

LeafyMind

5-Minute Loom Demo Script - Multi-Agent AI Concierge for Leafy Cave

Overview

Suggested video title: LeafyMind - AI Concierge for Leafy Cave (Multi-Agent + Rules + RAG). Target length: 4:45-5:15.
Before recording: docker compose up, seed data, Groq API key in .env, fresh guest account, <http://localhost:5174/hub> ready.

Pre-recording checklist

- docker compose up - all services healthy
- Seed packages, attractions, food (see README Quick start)
- New guest email ready (profile not yet complete)
- Food images in frontend/public/images/food/
- Terminal tab: docker compose ps + optional test command
- Browser zoom 100%, hide bookmarks bar

Minute-by-minute flow

Time	Section	On screen
0:00-0:35	Hook + problem	Landing or /hub
0:35-1:10	Architecture + stack	README diagram / docker ps
1:10-2:00	Register + Profile	Agent Hub -> Profile Builder
2:00-3:15	Three planners	Package -> Food -> Itinerary
3:15-4:15	Trip pack + tech	Trip pack PDF + terminal
4:15-5:00	Evals + close	test script + /owner mention

Section 1 - Hook (0:00-0:35)

Hi, I am [Your name]. This is LeafyMind - an AI-native pre-arrival concierge I built for Leafy Cave, a luxury cabana retreat in Wellawaya, Sri Lanka.

The real problem: international guests ask the same questions over and over on WhatsApp - which cabana package fits a couple versus a family, what to eat with dietary limits, how to plan days around Ella and waterfalls, and local culture like spice levels. The owner had no single shareable plan before guests fly.

LeafyMind fixes that with a guided multi-agent hub, business rules so packages do not hallucinate, and a branded trip-plan PDF guests can download or email.

ACTION: Show landing page -> click Agent Hub (/hub).

Section 2 - Architecture & technologies (0:35-1:10)

Under the hood this is a full production-style stack, not a ChatGPT wrapper.

Frontend: React 18, Vite, Tailwind - guest UI at /hub, /chat, and /owner.

BFF: Node Express - CORS, Helmet, rate limits, proxies /api; browser never sees API keys.

Backend: Python FastAPI, async SQLAlchemy, PostgreSQL 15.

AI layer: LangChain specialists on Groq through a single LLM gateway.

Multi-agent design - five Hub specialists plus classic orchestrator concierge on /chat. Agents do not call each other; orchestrator and agent runner route work.

Business rules engine - package matching in business_rules.py with scores and guards.

RAG / semantic retrieval - FAISS indexes over packages, food, attractions; sentence-transformers embeddings.

Structured artifacts - JSON per agent (profile, packages, food, itinerary) for UI cards and PDF.

External grounding - OpenTripMap, Unsplash, Gmail SMTP.

Everything runs in Docker Compose.

ACTION: Flash README architecture or docker compose ps for 5 seconds.

Section 3 - Profile Builder (1:10-2:00)

I register as a new guest - JWT auth, bcrypt passwords, role guest.

Step one is Profile Builder - an eight-step guided interview (tap-through). The LLM extracts preferences; the flow is structured: group type, travel style, budget, dietary needs, nights, and contact.

This profile is the source of truth. Package, food, and itinerary stay locked until profile completes.

Notice SSE streaming - Server-Sent Events over POST - so replies feel live.

ACTION: Register -> complete Profile Builder -> show planners unlock.

Section 4 - Package Planner (2:00-2:40)

Package Planner is where rules beat creativity. The system scores canonical packages like Love Nest Getaway and Together Time Package against the profile. There is a minimum strong-match score; below that we generate a custom package name.

Group-type guards stop obvious mistakes - romance packages do not surface for families. The LLM writes the warm narrative; the package list is rule-grounded.

ACTION: Open Package Planner -> show recommendations card / artifact sidebar.

Section 5 - Food Guide (2:40-3:05)

Food Guide combines semantic KB retrieval with structured output: must-try dishes, spice level, safe starters. Photos from local food library with Unsplash fallback. Guests see FoodGuideCard, not broken markdown in chat.

We fixed a real bug: artifacts over chat markdown for anything the UI must render.

ACTION: Show food cards with photos and spice labels.

Section 6 - Itinerary Planner (3:05-3:15)

Itinerary Planner blends curated PostgreSQL attractions with OpenTripMap discoveries near cabana coordinates. Structured day-by-day JSON feeds timeline UI and PDF.

When all three planners finish, the hub marks trip pack ready.

ACTION: Show itinerary timeline (2-3 days is enough).

Section 7 - Trip pack (3:15-4:00)

This is the MVP outcome: Your trip pack. One branded PDF - profile, packages, food with photos, itinerary - generated with fpdf2, not ask the LLM to make a PDF.

Guests download immediately or email my plan if SMTP is configured. Feedback survey email only fires after all three planners - separate triggers.

Workflow: Profile -> three specialists -> trip summary service -> PDF/email -> feedback -> owner dashboard on /owner.

ACTION: Download PDF -> scroll profile, food photo, itinerary. Mention email button.

Section 8 - Classic chat & evaluation (4:00-4:35)

Classic streaming concierge on /chat - multi-phase orchestrator: profiling, contact, recommending, itinerary, feedback. Same backend rules and KB, different UX.

We ship automated checks: Agent Hub flow script, package rule unit tests, auth smoke, verify external services. Package recommendations are regression-testable.

ACTION (optional): Flash /chat for 10 seconds.

```
docker exec leafymind-backend python -m scripts.test_agent_hub_flow
```

Section 9 - Close (4:35-5:00)

Security: secrets server-side, BFF rate limits, prompt sanitisation, JWT on API routes.

Built in a five-day forward-deploy trial using Cursor with project rules - no LLM calls from frontend, no agents calling agents, versioned SQL migrations.

LeafyMind turns repetitive owner WhatsApp work into a self-serve, rule-grounded, multi-agent journey with a document guests can pack. Repo and setup in README. Thanks for watching.

ACTION: End on /hub with trip pack visible or PDF cover.

Tech stack diagram

```

Guest -> React (Vite) -> Express BFF -> FastAPI -> PostgreSQL
      |
      v
    LangChain multi-agents (Groq) + orchestrator
      |
      v
Rules engine | FAISS RAG (MiniLM) | Artifacts JSON
      |
      v
OpenTripMap | Unsplash | Gmail SMTP -> Trip PDF
  
```

Bullet cheat sheet (pin next to monitor)

- Problem - WhatsApp overload, no shareable plan
- Stack - React -> BFF -> FastAPI -> Postgres, Docker
- Multi-agent - 5 hub agents + orchestrator; runner coordinates
- Rules - business_rules.py, scores, custom packages
- RAG - FAISS + sentence-transformers on packages/food/attractions
- Artifacts - JSON per agent -> UI cards + PDF
- SSE - streaming hub messages
- Integrations - OpenTripMap, Unsplash, Gmail
- Trip pack - fpdf2 PDF after 3 planners

- Tests - test_agent_hub_flow, package rules pytest
- Security - JWT, BFF limits, secrets server-side
- AI dev - Cursor rules, docs, eval scripts

Loom video description (paste under upload)

LeafyMind - AI concierge for Leafy Cave (Wellawaya, Sri Lanka).

Tech: React + Express BFF + FastAPI + PostgreSQL | LangChain multi-agents (Groq)
| Business rules engine | FAISS RAG (sentence-transformers) | SSE streaming
| OpenTripMap + Unsplash + Gmail | Branded trip PDF (fpdf2)

Demo path: /hub -> Profile -> Package / Food / Itinerary -> Trip pack PDF

Repo: [your GitHub URL]

Live: [your URL if deployed]

Recording tips

- Do not read every guided step - tap quickly and narrate structure.
- Pause 2 seconds on food photos and PDF scroll - visual proof beats jargon.
- Say "rules before LLM creativity" once - key differentiator.
- Run one test command in terminal - evaluators love it.
- If live demo breaks: cut to a PDF you already downloaded.